Assignment 3

Ramit K.C

Westcliff University

C Programming

Professor Niranjan Thakuri

April 9, 2026

Q1. Write a C program to find the maximum and minimum elements in an array.

```
#include <stdio.h>

int main(void){
    int n;
    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements: \n", n);
    for(int i = 0; i < n; i++){
        scanf("%d", &arr[i]);
    }

    int max = arr[0];
    int min = arr[0];

    for(int i = 1; i < n; i++){
        if(arr[i] > max){
            max = arr[i];
        }
        if(arr[i] < min){
            min = arr[i];
        }
    }

    printf("\nMaximum element: %d\n", max);
    printf("Minimum element: %d\n", min);

    return 0;
}
```

```
; if ($?) { gcc q1.c -o q1 } ; if ($?) {
Enter the size of array: 6
Enter 6 elements:
1 2 3 4 5 6

Maximum element: 6
Minimum element: 1
PS C:\Users\Lenovo\OneDrive\Desktop\c\assi
```

Q2. Implement a C program to reverse the elements of an array.

```c
#include <stdio.h>

int main(void){
    int n;
    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter %d elements: \n", n);
    for(int i = 0; i < n; i++){
        scanf("%d", &arr[i]);
    printf("The elements of array are:");
    for(int i = 0; i < n; i++){
        printf("%d ", arr[i]);
        printf("\n");
    }

    for(int i = 0; i < n / 2; i++){
        int temp = arr[i];
        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = temp;
    }

    printf("The reversed array is: ");
    for(int i = 0; i < n; i++){
        printf("%d ", arr[i]);
        printf("\n");
    }
    return 0;
}
}
```

Q3. Write a C program to sort an array of integers in ascending order using a sorting algorithm of your choice (e.g., bubble sort, selection sort, insertion sort).

```c
#include <stdio.h>

int main() {
    int n;
    printf("Enter the size of array: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: \n", n);
    for(int i = 0; i < n; i++){
        scanf("%d", &arr[i]);
    }
    // Bubble sort
    for(int i = 0; i < n-1; i++){
        for(int j = 0; j < n-i-1; j++){
            if(arr[j] > arr[j+1]){
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    printf("The elements of array in ascending order are: ");
    for(int i = 0; i < n; i++){
        printf("%d ", arr[i]);
    }
    return 0;
}
```

```
Enter the size of array: 3
Enter 3 elements:
1
2
3
The Focus folder in explorer (ctrl + click) order are: 1 2 3
```

Q4. Implement a C program to find the second largest element in an array.

```c
#include <stdio.h>
```

```c
int main() {
    int n;
    printf("Enter the number of elements in the array: ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter the elements of the array: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int largest = arr[0];
    int second_largest = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] > largest) {
            second_largest = largest;
            largest = arr[i];
        } else if (arr[i] > second_largest && arr[i] != largest) {
            second_largest = arr[i];
        }
    }

    printf("The second largest element in the array is: %d\n", second_largest);

    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd "c:\Us
 ; if ($?) { gcc q4.c -o q4 } ; if ($?) { .\q4 }
Enter the number of elements in the array: 1 2 3
Enter the elements of the array: The second largest element in the array is: 2
```

Q5. Write a C program to merge two sorted arrays into a single sorted array.

```c
#include <stdio.h>
int main() {
    int n1, n2;
    printf("Enter the size of first array: ");
    scanf("%d", &n1);
    int arr1[n1];
    printf("Enter %d elements for first array: \n", n1);
    for(int i = 0; i < n1; i++){
        scanf("%d", &arr1[i]);
    }
```

```c
    printf("Enter the size of second array: ");
    scanf("%d", &n2);
    int arr2[n2];
    printf("Enter %d elements for second array: \n", n2);
    for(int i = 0; i < n2; i++){
        scanf("%d", &arr2[i]);
    }

    int merged[n1 + n2];
    int i = 0, j = 0, k = 0;

    while(i < n1 && j < n2){
        if(arr1[i] < arr2[j]){
            merged[k++] = arr1[i++];
        } else {
            merged[k++] = arr2[j++];
        }
    }

    while(i < n1){
        merged[k++] = arr1[i++];
    }

    while(j < n2){
        merged[k++] = arr2[j++];
    }

    printf("Merged sorted array: ");
    for(int i = 0; i < n1 + n2; i++){
        printf("%d ", merged[i]);
    }

    return 0;
}
```

```
 ; if ($?) { gcc q5.c -o q5 } ; if ($?) { .\q5 }
Enter the size of first array: 2
Enter 2 elements for first array:
1 2
Enter the size of second array: 3
Enter 3 elements for second array:
1 2 3
Merged sorted array: 1 1 2 2 3
```

Q6. Write a C program to find the length of a string without using the built-in string functions.

```c
#include <stdio.h>

int main() {
    char abc[100];
    int count = 0;

    printf("Enter the string: ");
    scanf("%s", abc);

    for (int i =0; i<=100; i++) {
        if (abc[i] == '\0'){
            break;
        }
        count++;
    }

    printf("Length of string is %d.\n", count);

    return 0;
}
```

```
; if ($?) { gcc q6.c -o q6 } ; if ($?) { .\q6 }
Enter the string: qwerty
Length of string is 6.
```

Q7. Implement a C program to reverse a string.

```c
#include <stdio.h>
#include <string.h>
int main(void){
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    int length = 0;
    while(str[length] != '\0' && str[length] != '\n'){
        length++;
    }

    char reversed[length + 1];
    for(int i = 0; i < length; i++){
        reversed[i] = str[length - 1 - i];
    }
    reversed[length] = '\0';
    printf("Reversed string: %s\n", reversed);
```

```
    return 0;
}
; if ($?) { gcc q7.c -o q7 } ; if ($?) { .\q7 }
Enter a string: qwerty
Reversed string: ytrewq
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q8. Write a C program to check if a given string is a palindrome.

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int len, flag = 0;

    printf("Enter a string: ");
    scanf("%s", str);

    len = strlen(str);

    for (int i = 0; i < len / 2; i++) {
        if (str[i] != str[len - i - 1]) {
            flag = 1;
            break;
        }
    }

    if (flag == 0) {
        printf("The string is a palindrome.\n");
    } else {
        printf("The string is not a palindrome.\n");
    }

    return 0;
}
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> ca
 ; if ($?) { gcc q8.c -o q8 } ; if ($?) { .\q8 }
Enter a string: qwerty
The string is not a palindrome.
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q9. Implement a C program to count the occurrence of a specific character in a string.

```
#include <stdio.h>
#include <string.h>
int main(void){
    char str[100];
```

```c
    char ch;
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    printf("Enter a character to count: ");
    scanf("%c", &ch);

    int count = 0;
    for(int i = 0; str[i] != '\0' && str[i] != '\n'; i++){
        if(str[i] == ch){
            count++;
        }
    }
    printf("The character '%c' occurs %d times in the string.\n", ch, count);
    return 0;
}
```

```
Enter a string: qwe
Enter a character to count: q
The character 'q' occurs 1 times in the string.
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q10. Write a C program to concatenate two strings without using the built-in string functions.

```c
#include <stdio.h>
int main(void){
    char str1[100], str2[100], result[200];
    printf("Enter the first string: ");
    fgets(str1, sizeof(str1), stdin);
    printf("Enter the second string: ");
    fgets(str2, sizeof(str2), stdin);
    int i = 0, j = 0;
    while(str1[i] != '\0' && str1[i] != '\n'){
        result[i] = str1[i];
        i++;
    }
    while(str2[j] != '\0' && str2[j] != '\n'){
        result[i] = str2[j];
        i++;
        j++;
    }
    result[i] = '\0';
    printf("Concatenated string: %s\n", result);
    return 0;
}
```

```
Enter the first string: q
Enter the second string: w
Concatenated string: qw
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q11. Write a function named calculateAverage that takes an array of integers as input and returns the average of the numbers.

```c
#include <stdio.h>
double calculateAverage(int arr[], int size) {
    if (size == 0) {
        return 0.0; // Avoid division by zero
    }
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return (double)sum / size;
}
int main(void) {
    int numbers[100];
    int size;
    printf("Enter the number of elements: ");
    scanf("%d", &size);
    if (size > 100) {
        printf("Size exceeds the maximum limit of 100.\n");
        return 1;
    }
    printf("Enter %d integers:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &numbers[i]);
    }
    double average = calculateAverage(numbers, size);
    printf("The average is: %.2f\n", average);
    return 0;
}
```

```
; if ($?) { gcc q11.c -o q11 } ; if ($?) { .\q11 }
Enter the number of elements: 3
Enter 3 integers:
1 2 3
The average is: 2.00
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q12. Write a function named isPalindrome that takes a string as input and returns 1 if it is a palindrome (reads the same forwards and backwards), and 0 otherwise.

```c
#include <stdio.h>
#include <string.h>
int isPalindrome(char str[]) {
    int length = 0;
    while(str[length] != '\0' && str[length] != '\n'){
        length++;
    }
    for(int i = 0; i < length / 2; i++){
        if(str[i] != str[length - 1 - i]){
            return 0; // Not a palindrome
        }
    }
    return 1; // Is a palindrome
}
int main(void) {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    if (isPalindrome(str)) {
        printf("The string is a palindrome.\n");
    } else {
        printf("The string is not a palindrome.\n");
    }
    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd
 ; if ($?) { gcc q12.c -o q12 } ; if ($?) { .\q12 }
Enter a string: qwe
The string is not a palindrome.
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q13. Write a function named findFactorial that takes an integer as input and returns its factorial.

```c
#include <stdio.h>

int findFactorial(int num) {
    int i = 1, fact = 1;

    while (i <= num) {
        fact *= i;
        i++;
    }
```

```
    return fact;
}

int main() {
    int num;

    printf("Enter the number: ");
    scanf("%d", &num);

    int factorial = findFactorial(num);

    printf("The factorial is %d.\n", factorial);

    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd
 ; if ($?) { gcc q13.c -o q13 } ; if ($?) { .\q13 }
Enter the number: 4
The factorial is 24.
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q14. Write a function named convertTemperature that takes a temperature value in Celsius and converts it to Fahrenheit. The function should return the converted temperature.

```
#include <stdio.h>
double convertTemperature(double celsius) {
        return (celsius * 9.0 / 5.0) + 32;
}
   int main(void) {
       double celsius;
       printf("Enter temperature in Celsius: ");
       scanf("%lf", &celsius);
     double fahrenheit = convertTemperature(celsius);
     printf("%.2f Celsius is equal to %.2f Fahrenheit.\n", celsius, fahrenheit);
     return 0;
 }
```

```
  ; if ($?) { gcc q14.c -o q14 } ; if ($?) { .\q14 }
 Enter temperature in Celsius: 122
 122.00 Celsius is equal to 251.60 Fahrenheit.
 PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q15. Write a function named countOccurrences that takes a string and a character as input and returns the number of times the character appears in the string.

```
#include <stdio.h>
#include <string.h>
 #include <stdio.h>
  int countOccurrences(char str[], char ch) {
    int count = 0;
    for (int i = 0; str[i] != '\0' && str[i ] != '\n'; i++) {
          if (str[i] == ch) {
              count++;
          }
      }
      return count;
  }
  int main(void) {
      char str[100];
      char ch;
      printf("Enter a string: ");
      fgets(str, sizeof(str), stdin);
      printf("Enter a character to count: ");
      scanf("%c", &ch);
      int occurrences = countOccurrences(str, ch);
      printf("The character '%c' occurs %d times in the string.\n", ch
  , occurrences);
      return 0;
  }
```

```
 ; if ($?) { gcc q15.c -o q15 } ; if ($?) { .\q15 }
Enter a string: uiu
Enter a character to count: y
The character 'y' occurs 0 times in the string.
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q16. Write a function named reverseArray that takes an array of integers as input and reverses the order of the elements in the array.

```
#include <stdio.h>
  void reverseArray(int arr[], int size) {
      for (int i = 0; i < size / 2; i++) {

         int temp = arr[i];
          arr[i] = arr[size - 1 - i];
          arr[size - 1 - i] = temp;
      }
   }
```

```c
int main(void) {
    int numbers[100];
    int size;
    printf("Enter the number of elements: ");
    scanf("%d", &size);
    if (size > 100) {
        printf("Size exceeds the maximum limit of 100.\n");
        return 1;
    }
    printf("Enter %d integers:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &numbers[i]);
    }
    reverseArray(numbers, size);
    printf("Reversed array:\n");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");
    return 0;
}
```

```
; if ($?) { gcc q16.c -o q16 } ; if ($?) { .\q16 }
Enter the number of elements: 2
Enter 2 integers:
1 2
Reversed array:
2 1
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q17. Write a function named calculatePower that takes two integers, base and exponent, as input and returns the result of raising the base to the exponent.

```c
#include <stdio.h>

int calculatePower(int base, int exponent) {
    int result = 1;

    for (int i = 0; i < exponent; i++) {
        result *= base;
    }

    return result;
}

int main() {
```

```
    int b, e;

    printf("Enter base and exponent: ");
    scanf("%d %d", &b, &e);

    int power = calculatePower(b, e);

    printf("Result is %d.\n", power);

    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cc
 ; if ($?) { gcc q17.c -o q17 } ; if ($?) { .\q17 }
Enter base and exponent: 2 3
Result is 8.
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q18. Write a function named findPrimeNumbers that takes an integer n as input and prints all prime numbers from 1 to n.

```
#include <stdio.h>
  int isPrime(int num) {
    if (num <= 1) {
        return 0; // Not prime
     }
     for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0) {
            return 0; // Not prime
        }
     }
     return 1; // Is prime
  }
  void findPrimeNumbers(int n) {
    printf("Prime numbers from 1 to %d are:\n", n);
    for (int i = 1; i <= n; i++) {
        if (isPrime(i)) {
            printf("%d ", i);
        }
    }
    printf("\n");
  }
  int main(void) {
    int n;
    printf("Enter an integer: ");
    scanf("%d", &n);
```

```
    findPrimeNumbers(n);
    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd "c:\
 ; if ($?) { gcc q18.c -o q18 } ; if ($?) { .\q18 }
Enter an integer: 3
Prime numbers from 1 to 3 are:
2 3
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q19. Write a function named calculateFactorialSeries that takes an integer n as input and prints the factorial series up to n.

```
#include <stdio.h>
 unsigned long long factorial(int n) {
    if (n < 0) {
        return 0; // Factorial is not defined for negative numbers
    }
    unsigned long long result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
 }
 void calculateFactorialSeries(int n) {
    printf("Factorial series up to %d:\n", n);
    for (int i = 0; i <= n; i++) {
        printf("%d! = %llu\n", i, factorial(i));
    }
 }
 int main(void) {
    int n;
    printf("Enter an integer: ");
    scanf("%d", &n);
    calculateFactorialSeries(n);
    return 0;
 }
```

```
 ; if ($?) { gcc q19.c -o q19 } ; if ($?) { .\q19 }
Enter an integer: 3
Factorial series up to 3:
0! = 1
1! = 1
2! = 2
3! = 6
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q20. Write a function named calculateGCD that takes two integers as input and returns their greatest common divisor (GCD).

```
#include <stdio.h>
  int calculateGCD(int a, int b) {
     while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
     }
     return a;
  }
  int main(void) {
    int num1, num2;
     printf("Enter the first integer: ");
     scanf("%d", &num1);
     printf("Enter the second integer: ");
     scanf("%d", &num2);
     int gcd = calculateGCD(num1, num2);
     printf("The GCD of %d and %d is: %d\n", num1, num2, gcd);
    return 0;
  }
```

```
 ; if ($?) { gcc q20.c -o q20 } ; if ($?) { .\q20 }
Enter the first integer: 1
Enter the second integer: 2
The GCD of 1 and 2 is: 1
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q21. Write a recursive function named calculateFactorial that takes an integer n as input and returns its factorial.

```
#include <stdio.h>
  unsigned long long calculateFactorial(int n) {
     if (n < 0) {
        return 0; // Factorial is not defined for negative numbers
     }
     if (n == 0 || n == 1) {
        return 1; // Factorial of 0 and 1 is 1
     }
     return n * calculateFactorial(n - 1); // Recursive call
  }
  int main(void) {
     int number;
     printf("Enter an integer: ");
     scanf("%d", &number);
```

```
    unsigned long long result = calculateFactorial(number);
    if (result == 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        printf("The factorial of %d is: %llu\n", number, result);
    }
    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd "c:\U
 ; if ($?) { gcc q21.c -o q21 } ; if ($?) { .\q21 }
Enter an integer: 4
The factorial of 4 is: 24
```

Q22. Write a recursive function named calculateFibonacci that takes an integer n as input and returns the nth Fibonacci number. The Fibonacci sequence starts with 0 and 1, and each subsequent number is the sum of the two preceding numbers.

```
#include <stdio.h>
unsigned long long calculateFibonacci(int n) {
    if (n < 0) {
        return 0; // Fibonacci is not defined for negative numbers
    }
    if (n == 0) {
        return 0; // Fibonacci of 0 is 0
    }
    if (n == 1) {
        return 1; // Fibonacci of 1 is 1
    }
    return calculateFibonacci(n - 1) + calculateFibonacci(n - 2);
}
int main(void) {
    int n;
    printf("Enter an integer: ");
    scanf("%d", &n);
    unsigned long long result = calculateFibonacci(n);
    if (n < 0) {
        printf("Fibonacci is not defined for negative numbers.\n");
    } else {
        printf("The %dth Fibonacci number is: %llu\n", n, result);
    }
    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd
 ; if ($?) { gcc q22.c -o q22 } ; if ($?) { .\q22 }
Enter an integer: 3
The 3th Fibonacci number is: 2
```

Q23. Write a recursive function named calculateGCD that takes two integers a and b as input and returns their greatest common divisor (GCD).

```c
#include <stdio.h>
int calculateGCD(int a, int b) {
    if (b == 0) {
        return a; // GCD is found
    }
    return calculateGCD(b, a % b); // Recursive call
}
int main(void) {
    int num1, num2;
    printf("Enter the first integer: ");
    scanf("%d", &num1);
    printf("Enter the second integer: ");
    scanf("%d", &num2);
    int gcd = calculateGCD(num1, num2);
    printf("The GCD of %d and %d is: %d\n", num1, num2, gcd);
    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd
 ; if ($?) { gcc q23.c -o q23 } ; if ($?) { .\q23 }
Enter the first integer: 1
Enter the second integer: 2
The GCD of 1 and 2 is: 1
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q24. Write a recursive function named calculatePower that takes two integers base and exponent as input and returns the result of raising the base to the exponent.

```c
#include <stdio.h>
double calculatePower(int base, int exponent) {
    if (exponent < 0) {
        return 1 / calculatePower(base, -exponent); // Handle negative exponent
    }
    if (exponent == 0) {
        return 1; // Any number raised to the power of 0 is 1
    }
    return base * calculatePower(base, exponent - 1); // Recursive call
}
int main(void) {
```

```c
    int base, exponent;
    printf("Enter the base: ");
    scanf("%d", &base);
    printf("Enter the exponent: ");
    scanf("%d", &exponent);
    double power = calculatePower(base, exponent);
    printf("%d raised to the power of %d is: %.2f\n", base, exponent, power);
    return 0;
}
```

```
; if ($?) { gcc q24.c -o q24 } ; if ($?) { .\q24 }
Enter the base: 2
Enter the exponent: 2
2 raised to the power of 2 is: 4.00
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q25. Write a recursive function named calculateSumOfDigits that takes an integer n as input and returns the sum of its digits.

```c
#include <stdio.h>
int calculateSumOfDigits(int n) {
    if (n < 0) {
        n = -n;
    }
    if (n == 0) {
        return 0;
    }
    return (n % 10) + calculateSumOfDigits(n / 10); //
}
int main(void) {
    int number;
    printf("Enter an integer: ");
    scanf("%d", &number);
    int sum = calculateSumOfDigits(number);
    printf("The sum of the digits of %d is: %d\n", number, sum);
    return 0; }
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd "c:
 ; if ($?) { gcc q25.c -o q25 } ; if ($?) { .\q25 }
Enter an integer: 3
The sum of the digits of 3 is: 3
```

Q26. Write a recursive function named reverseString that takes a string as input and returns the reversed string.

```c
#include <stdio.h>
#include <string.h>
```

```
void reverseString(char str[], int start, int end) {
    if (start >= end) {
        return;
    }

    char temp = str[start];
    str[start] = str[end];
    str[end] = temp;

    reverseString(str, start + 1, end - 1);
}
int main(void) {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    // Remove newline character if present
    size_t length = strlen(str);
    if (length > 0 && str[length - 1] == '\n') {
        str[length - 1] = '\0';
    }
    reverseString(str, 0, strlen(str) - 1);
    printf("Reversed string: %s\n", str);
    return 0;
}
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd "c:\Us
 ; if ($?) { gcc q26.c -o q26 } ; if ($?) { .\q26 }
Enter a string: qwerty
Reversed string: ytrewq
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q27. Write a recursive function named printTriangle that takes an integer n as input and prints a triangle of asterisks (*) with n rows.

```
#include <stdio.h>
void printTriangle(int n) {
    if (n <= 0) {
        return; // Base case: when n is 0 or negative, stop recursion
    }
    printTriangle(n - 1); // Recursive call to print the upper part of the triangle
    for (int i = 0; i < n; i++) {
        printf("* ");
    }
    printf("\n"); // Move to the next line after printing each row
}
int main(void) {
    int n;
```

```
    printf("Enter the number of rows for the triangle: ");
    scanf("%d", &n);
    printTriangle(n);
    return 0;}
```

```
; if ($?) { gcc q27.c -o q27 } ; if ($?) { .\q27 }
Enter the number of rows for the triangle: 4 5
*
* *
* * *
* * * *
```

Q28. Write a recursive function named calculateBinary that takes an integer n as input and returns its binary representation as a string.

```
#include <stdio.h>
  #include <string.h>
  void calculateBinary(int n, char binaryStr[], int index) {
      if (n == 0) {
          binaryStr[index] = '\0'; // Null-terminate the string
          return; // Base case: when n is 0, stop recursion
      }
      calculateBinary(n / 2, binaryStr, index - 1); // Recursive call for the next bit
     binaryStr[index] = (n % 2) + '0'; // Convert the bit to a character and store it
  }
  int main(void) {
     int number;
     printf("Enter an integer: ");
    scanf("%d", &number);
     char binaryStr[32]; // Assuming a 32-bit integer
     calculateBinary(number, binaryStr, 31); // Start from the last index
     printf("The binary representation of %d is: %s\n", number, binaryStr
    );
     return 0;
  }
```

```
; if ($?) { gcc q28.c -o q28 } ; if ($?) { .\q28 }
Enter an integer: 3
The binary representation of 3 is: a
```

Q29. Write a recursive function named isPalindrome that takes a string as input and returns 1 if it is a palindrome (reads the same forwards and backwards), and 0 otherwise.

```
#include <stdio.h>
  #include <string.h>
  int isPalindrome(char str[], int start, int end) {
```

```
    if (start >= end) {
        return 1; // Base case
    }
    if (str[start] != str[end]) {
        return 0; // Not a palindrome
    }
    return isPalindrome(str, start + 1, end - 1); // Recursive call
}
int main(void) {
    char str[100];
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
    size_t length = strlen(str);
    if (length > 0 && str[length - 1] == '\n') {
        str[length - 1] = '\0';
    }
    if (isPalindrome(str, 0, strlen(str) - 1)) {
        printf("The string is a palindrome.\n");
    } else {
        printf("The string is not a palindrome.\n");
    }
    return 0;
}
```

```
The binary representation of 3 is: a
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src> cd "c
 ; if ($?) { gcc q29.c -o q29 } ; if ($?) { .\q29 }
Enter a string: qwerty
The string is not a palindrome.
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```

Q30. Write a recursive function named countOccurrences that takes a string and a character as input and returns the number of times the character appears in the string.

```
#include <stdio.h>
int countOccurrences(char str[], char ch, int index) {
    if (str[index] == '\0' || str[index] == '\n') {
        return 0; // Base case: end of string
    }
    int count = (str[index] == ch) ? 1 : 0;
    return count + countOccurrences(str, ch, index + 1);
}
int main(void) {
    char str[100];
    char ch;
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);
```

```c
    printf("Enter a character to count: ");
    scanf("%c", &ch);
    int occurrences = countOccurrences(str, ch, 0); // Start from index 0
    printf("The character '%c' occurs %d times in the string.\n", ch, occurrences);
    return 0;
}
```

```
; if ($?) { gcc q30.c -o q30 } ; if ($?) { .\q30 }
Enter a string: trewty
Enter a character to count: e
The character 'e' occurs 1 times in the string.
PS C:\Users\Lenovo\OneDrive\Desktop\c\assignment-3-ramitkc750-del\src>
```